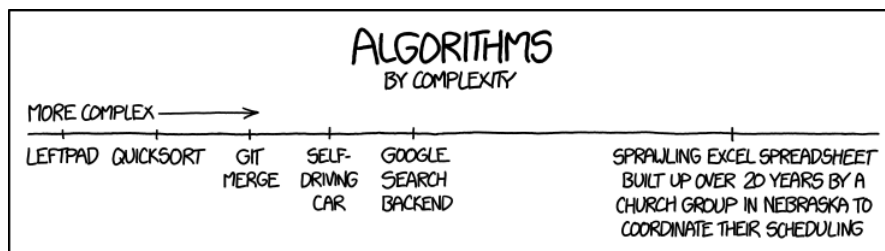


Chapitre 8

Complexité



On établit les relations suivantes :

Problème de décision $\leftrightarrow$ donnée $f : D \rightarrow \mathbb{B}$.

Si le problème est décidable, on peut étudier :

- Temporelle
- Spatiale (HP)

On s'intéresse donc à **la complexité des problèmes**, i.e la quantité de temps nécessaire à sa résolution, une quantité suffisante étant facilement mise en évidence par un algorithme donné.

1 Notion de complexité temporelle

Définition - Opérations élémentaires

Une opération élémentaire est une opération :

- lecture, écriture
- arithmétiques sur les entiers machines (ceux de valeur $\leq 2^{63} - 1$)

Définition - Taille des entrées

La taille d'une entrée est l'espace mémoire utilisée par la variable.

Pour les tableaux et les listes, on prendra la longueur.

Si l'entrée est $n \in \mathbb{N}$ alors $|n| = \mathcal{O}(\log(n))$

Exemple : Soit $G = (S, A)$ un graphe, **la taille dépend de la représentation :**

- mat d'adja : $\mathcal{O}(|S|^2)$
- liste d'adja : $\mathcal{O}(|S| + |A|) = \mathcal{O}(|S|^2)$
- liste des arêtes : $\mathcal{O}(|A|) = \mathcal{O}(|S|^2)$

Définition - Complexité d'un algo

Un algorithme (**ou un programme**) a une complexité en :

$\mathcal{O}(t(n))$ (avec $t : \mathbb{N} \rightarrow \mathbb{N}$)

si il fait $\mathcal{O}(t(n))$ opérations élémentaires au pire cas sur une entrée de taille n.

Attention : si on traite d'un problème de décision sur $\mathbb{N}$, on ne se contente pas de la restriction à la capacité machine et donc les opérations et la représentation ne sont plus en temps ni en espace constant.

Définition - Complexité d'un problème

Soit $f : D \rightarrow \mathbb{B}$ et le P le problème de décision associé alors on dit que $P \in DTIME(t(n))$ (avec $t : \mathbb{N} \rightarrow \mathbb{N}$) s'il existe un algo qui résout le pb P avec une complexité $\mathcal{O}(n)$ où n est la taille de l'entrée du pb.

2 La classe P

Définition

Un problème A associé à $f : D \rightarrow \mathbb{B}$ est dans la classe P si il existe une fonction polynomiale $t : \mathbb{N} \rightarrow \mathbb{N}$ tel que $A \in DTIME(t(n))$

Remarques :

- C'est bien un problème qui est dans la classe P
- Cette classe n'a de sens que pour les problèmes de décision
- On peut facilement imbriquer, composer des problèmes de la classe P en y restant
- On considère en général (mais c'est pas si vrai) que les problèmes de la classe P sont ceux pour lesquels il existe des programmes dont le tps d'exécution est raisonnable.

Exemples :

- La recherche d'un élément dans une liste

- La recherche des CC (ou CFC) dans un graphe
- Tester si un langage reconnaissable est vide
- 2-SAT
- tester si deux nombres sont premiers entre eux (algorithme d'Euclide)

Exercice

Montrer par récurrence sur n que si l'algorithme d'euclide appliqué aux nombres a et b fait n divisions alors $a \geq F_{n+2}$ et $b \geq F_{n+1}$

3 La classe NP

Définition

Un problème A associé à $f : D \rightarrow \mathbb{B}$, A est dit vérifiable si :

$$\exists v : D \times \Sigma^* \rightarrow \mathbb{B}, \forall x \in D, f(x) \text{ est vrai ssi } \exists c \in \Sigma^* \text{ tq. } v(x, c) = \mathbf{true}$$

Définition

Un problème A est dans la classe NP ssi

- il existe un vérificateur v pour A avec le problème de décision associé à v qui est dans P
- pour chaque instance positive de A , il existe un certificat c_x t.q. $|c_x|$ est polynomial en $|x|$

Exemple fondamental n°1

$$\text{SAT} \in \text{NP}$$

DÉMONSTRATION.

- On définit $v : \phi \text{ formule du calcul prop, } \langle \sigma \rangle \rightarrow \mathbf{true} \Leftrightarrow \sigma \models \phi$ avec $\langle \sigma \rangle$ une valuation.
- À l'aide d'un parcours inductif de ϕ , on peut résoudre le problème v en temps linéaire en la taille de ϕ .
- Pour chaque instance ϕ positive, il existe un valuation σ qui est une fonction booléenne sur l'ensemble des variables de ϕ et donc $|\langle \sigma \rangle|$ est polynomiale en ϕ .
- $\phi \in \text{SAT} \Leftrightarrow (\exists \sigma, \sigma \models \phi \Leftrightarrow c \in \Sigma^* \text{ tq. } c = \langle \sigma \rangle \text{ tq. } v(\phi, c) = \mathbf{true})$

Exemple fondamental n°2

$$\text{Pour tout } k \in \mathbb{N}, k \geq 2 : k\text{-color} \in \text{NP}$$

DÉMONSTRATION.

- Color : $\langle G, k \rangle \Rightarrow$ est-ce qu'il existe un coloriage valide de G avec k couleurs.
- On définit $v : \langle G, k \rangle, \langle c \rangle \rightarrow \mathbf{true}$ si c représente bien un k -coloriage de G (revoir définition).

Avec $\langle c \rangle$ la description d'un coloriage, i.e l'encodage d'un tableau d'entiers quelconque

- Le problème associé à v est dans la classe P
- La description d'un coloriage pour G est polynomiale en $|G|$ car il suffit d'un tableau dont la taille est le nombre de sommets de G
- $\langle G, k \rangle \in \text{Color ssi} : (\text{il existe un coloriage pour } G \text{ de taille } k \Leftrightarrow \exists \in \Sigma^* \text{ tq. } v(\langle G, k \rangle, c) = \mathbf{true})$

Autres exemples

- $G \rightarrow$ est-ce qu'il existe un chemin hamiltonien dans G

Remarque : la classe NP ne concerne que les problèmes de décision

Théorème

$$P \subset NP$$

DÉMONSTRATION.

Soit A un problème de classe P , i.e il existe un algorithme a qui résout A en temps polynomial.

On définit $v : D \times \Sigma^* \rightarrow \mathbb{B}$, $v(x, c) \mapsto f(x)$.

v est dans la classe P

On prend x et c en entrée et on applique a à x

Cet algorithme résout v en temps polynomial. $x \in A \Leftrightarrow f(x) = \mathbf{vrai} \Leftrightarrow \exists c \in \Sigma^*$ tq. $|c|$ est polynomail en $|x|$ et $v(x, c) = \mathbf{vrai}$ (par exemple en prenant $x = c$)

4 Réduction polynomiale

Définition

Soient $f_1 : D_1 \rightarrow \mathbb{B}$, $f_2 : D_2 \rightarrow \mathbb{B}$ associés à A_1 et A_2 deux problèmes de décision, on dit que $A_1 \leq_p A_2$ ssi :

$\exists g : D_1 \rightarrow D_2$ tq. $\forall x \in D_1$, $f_1(x) = f_2(g(x))$ et il existe un algorithme de complexité polynomiale en $|x|$ qui calcule $g(x)$

Théorème

Si $A_1 \leq_p A_2$ et $A_2 \in P$ alors $A_1 \in P$

DÉMONSTRATION.

Supposons $A_1 \leq_p A_2$, $A_2 \in P$.

Soit g réalisant la réduction. Considérons l'algo suivant :

Entrée : $x \in D_1$

- calculer $g(x)$
- exécuter un algorithme polynomial a_2 résolvant f_2 sur l'entrée $g(x)$

Cet algorithme calcule $f_2(g(x)) = f_1(x)$ et donc résout A_1 .

→ **Justifions que cet algorithme est polynomial**

Supposons qu'on sait calculer $g(x)$ à partir de x avec un algo

en $\mathcal{O}(t_g|x|)$ et que a_2 a une complexité en $\mathcal{O}(t_2(n))$. Notre algo a alors une complexité en

$$\mathcal{O}(t_g|x| + t_2(|g(x)|)) = \mathcal{O}(t_g|x| + t_2(t_g(|x|)))$$

car $|g(x)| \leq t_g(|x|)$. En effet, dans l'algo pour g on a au moins $|g(x)|$ étapes pour écrire chaque bit de $g(x)$ dans la mémoire.

Ainsi, avec t_2 et t_g polynomiales on a bien le résultat voulu.

Exemple fondamental n°3

$$3SAT \leq_p \text{Clique}$$

DÉMONSTRATION.

- **Clique :** $\langle G, k \rangle \rightarrow$ est-ce qu'on peut trouver k sommets de G tous reliés 2 à 2.
- **3-SAT :** φ sous forme normale conjonctive dont toutes les clauses sont de taille 3 $\rightarrow$ est-ce que φ est satisfiable?

Soit $\varphi = C_1 \wedge \dots \wedge C_k$ avec $\forall i \in [1; k]$, $C_i = l_1^i \vee l_2^i \vee l_3^i$

On construit G_φ de la manière suivante : Pour chaque clause, on crée 3 sommets correspondants aux trois littéraux avec aucune arête entre eux. De plus, on relie ces sommets avec les sommets des autres clauses, sauf s'il s'agit de leur négation.

Exemple avec $\varphi = (x_1 \vee x_2 \vee \neg x_3) \wedge (x_1 \vee x_3 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$

Insérer dessin.

On remarque qu'une clique de G a au plus k elts (au plus un elt dans chaque clause) et qu'une clique ne peut pas contenir un littéral & et sa négation (par construction de G_φ).
 Montrons que $\varphi \in SAT \Leftrightarrow \langle G, k \rangle \in Clique$:

□ Posons

$$g : \varphi = C_1 \wedge \dots \wedge C_k \mapsto \langle G_\varphi, k \rangle$$

□ Le calcul de $g(\varphi)$ est bien en temps polynomial en $|\varphi|$.

□ Il reste à prouver que

$$\varphi \text{ sat.} \Leftrightarrow G_\varphi \text{ admet une clique de taille } k.$$

$\Leftarrow$: Supposons que G_φ admette une clique de taille k .

Dans ce cas, il y a exactement un sommet de chaque clause qui est dans la clique.

On définit une valuation σ en mettant à **true** les littéraux qui composent cette clique, et à **false** toutes les autres variables dont la valuation n'a pas été définie.

Le fait étant que l'on ne peut avoir une variable et sa négation dans la même clique car on a construit G_φ tel que ce genre d'arêtes existent (et donc si une variable et sa négation était dans la même clique, elles auraient une arête les reliant et ce ne serait plus une clique, ce qui est absurde). Ainsi, σ est bien définie.

Il y a donc un sommet de chaque clause évalué à vrai dans chaque clause de φ .

φ est bien satisfiable.

$\Rightarrow$: Supposons que φ soit satisfiable.

5 Problèmes NP-complet

Définition

Un pb A est dit NP-complet ssi

- Il est dans NP
- Pour tout $B \in NP$, on a $B \leq_p A$

Proposition

Si il existe un problème C NP-complet et dans P alors $P = NP$

DÉMONSTRATION.

On sait déjà que $P \subset NP$. Montrons l'inclusion réciproque.

Soit $A \in NP$, par (2) (C étant NP-complet) on a $A \leq_p C \in P$.

Ainsi, par propriété de la réduction polynomiale, $A \in P$

Proposition

Soit A un pb de la classe NP et B un problème NP-complet tq $B \leq_p A$, alors A est NP-complet.

DÉMONSTRATION.

Soit $A \in NP$ (1). Soit $C \in NP$, comme B est NP-complet $C \leq_p B$ or $B \leq_p A$ donc $C \leq_p A$.

Justifions que $\leq_p$ est transitive.

Soit $C \leq_p B$ et $B \leq_p A$, i.e $\exists f : D_C \rightarrow D_B$ et $g : D_B \rightarrow D_A$ tq :

- $\forall x \in D_C, x$ est une instance positive de $C \Leftrightarrow f(x)$ est une instance positive de B
- $\forall y \in D_B, y$ est une instance positive de $B \Leftrightarrow g(y)$ est une instance positive de A

Soit $x \in D_C$,

$$\begin{aligned} x \text{ est une instance positive de } C &\Leftrightarrow f(x) \text{ est une instance positive de } B \\ &\Leftrightarrow g \circ f(x) \text{ est une instance positive de } A \end{aligned}$$

donc $g \circ f$ est une fonction calculable en temps polynomiale qui met en évidence que $C \leq_p A$

Méthode pour montrer que A est NP-complet

- Justifier que $A \in NP$
- Trouver B NP-complet et montrer que $B \leq_p A$

Théorème - Cook-Levin

SAT est NP-complet

Remarque : à partir de maintenant, ce sont les preuves et non pas les énoncés qui sont exigibles (cf. Garey-Johnson).

Théorème

3-SAT est NP-complet

DÉMONSTRATION.

- 3-SAT $\in NP$ car une description d'une valuation est de taille polynomiale en la formule et tester si une telle valuation satisfait bien la formule se fait en temps polynomial et une telle valuation existera ssi la formule est satisfiable.
- Soit $g : \varphi \rightarrow \tilde{\varphi}$, avec φ quelconque du calcul propositionnel (se limitant à $\wedge, \vee$ et $\neg$) et $\tilde{\varphi}$ sous forme CNF avec des clauses de taille exactement 3, t.q $\varphi \in SAT \Leftrightarrow \tilde{\varphi} \in 3-SAT$

- Pour passer sous forme CNF-équivalente, on dresse une table de vérité : pour ψ quelconque, à partir de la table de vérité de ψ , on obtient directement ψ' équivalente sous forme DNF. En obtenant la DNF de $\neg\varphi$ à l'aide de sa table de vérité puis en utilisant les lois de Morgan, on obtient la CNF de φ .

Problème : la transformation de φ en φ' se fait en temps exponentiel donc il faut une transformation plus fine. (cf DM transformation de Tseitin). On admet donc provisoirement que $SAT \leq_p SAT - CNF$.

- Montrons que $SAT - CNF \leq_p 3 - SAT$:

Soit $\varphi = C_1 \wedge \dots \wedge C_k$. Sous forme CNE, pour chaque clause C_i :

- Si elle est de taille 1 ou 2, on répète 1 ou 2 littéral pour obtenir une clause de taille 3.
- Si elle est de taille au moins 4 :

$$C_i = a_1 \vee \dots \vee a_n \text{ avec } n \geq 4$$

On construit

$$F = (a_1 \vee a_2 \vee b_3) \wedge (\neg b_3 \vee a_3 \vee b_4) \wedge (\neg b_4 \vee a_4 \vee b_5) \wedge \dots \wedge (\neg b_{n-1} \vee a_{n-1}) \vee a_n$$

où les $(b_i)_{1 \leq i \leq n-1}$ sont des variables nouvelles.

Montrons que C est satisfiable ssi F l'est.

- Soit $\sigma \models F$ alors on va montrer que $\exists i \in [1; n]$, tq $\sigma(a_i) = \text{true}$ et donc $\sigma_{\{a_1, \dots, a_n\}} \models C$.
Supposons que σ est une valuation rendant tous les a_i faux. Si $\sigma \models F$ alors $\sigma(b_3) = \text{true}$ puis par rec. immédiate on a nécessairement $\forall i \in [3; n-1], \sigma(b_i) = \text{true}$ et ainsi la dernière clause n'est pas satisfaite.
- Réciproquement, supposons C satisfiable. Soit $\sigma \models C$, on va construire $\tilde{\sigma} \models F$. Prenons $\forall i \in [1; n], \tilde{\sigma}(a_i) = \sigma(a_i)$.
Soit i_0 le plus petit indice tq $\sigma(a_{i_0}) = \text{true}$. On pose $\tilde{\sigma}(b_{i_0}) = \text{true}$ et même $\forall k$ tq. $3 \leq k \leq i_0, \tilde{\sigma}(b_k) = \text{true}$.
et $\forall k$ tq. $i_0 + 1 \leq k \leq n-1, \tilde{\sigma}(b_k) = \text{false}$, on aura $\tilde{\sigma} \models F$ car

$$\forall k \text{ tq. } 3 \leq k \leq i_0, \tilde{\sigma} \models (\neg b_{k-1} \vee a_{k-1} \vee b_k)$$

$$\text{et } \tilde{\sigma} \models (\neg b_{i_0} \vee a_{i_0} \vee b_{i_0+1})$$

$$\text{et } \forall k \text{ tq. } i_0 + 1 \leq k \leq n-1, \tilde{\sigma} \models (\neg b_k \vee a_k \vee b_{k+1})$$

On procède ainsi pour chaque clause trop grande en utilisant des nouvelles variables différentes pour chaque clause.

On remarque que la construction est linéaire en la taille de la formule initiale.

Théorème

Clique est NP-Complet

DÉMONSTRATION.

- Clique $\in$ NP :
 - On propose d'utiliser comme certificat un ensemble de sommets du graphe.
 - On prend $\nu : (G, k), \langle S' \rangle \mapsto \text{vrai}$ ssi S' est bien de cardinal $\geq k$ et pour tout $(x, y) \in S^2, x \neq y \Rightarrow (x, y) \in A$
 - $\nu \in P$ car se fait en $\mathcal{O}(|S'| + |S'|^2 \times |G|)$
 - Si $\langle G, k \rangle \in \text{Clique}$ alors $\exists S' \subset S$ t.q. $\#S' \geq k$ et S' est une clique ds G donc $\nu((G, k), \langle S' \rangle) = \text{vrai}$ et $|\langle S' \rangle|$ est polynomiale en $|G|$ (par exemple un tableau de booléens de taille $\#S = \mathcal{O}(|G|)$).
 - Si $\langle G, k \rangle \notin \text{Clique}$ alors $\forall c \in \{0, 1\}^*, \nu((G, k), c) = \text{faux}$.
- 3-SAT est NP-complet et $3\text{-SAT} \leq_p \text{Clique}$: (cf. preuves précédentes)

Finalement, Clique est bien NP-complet.

On a déjà vu que $3 - SAT \leq_p \text{Clique}$ donc Clique est NP-complet

Théorème

Soit le problème **Independant Set** :

Entrée : $\langle G, k \rangle$

Instance : existe-t-il un ensemble de k sommets non reliés entre eux, i.e n'ayant aucune arête les reliant.

Independant Set est NP-complet

DÉMONSTRATION.

- Ce problème est dans NP car si on donne un ensemble de sommets on peut vérifier en temps polynomiale son cardinal et le fait qu'ils sont non reliés 2 à 2.
- $\text{Clique} \leq_p \text{IS}$ en prenant $f : \langle G, k \rangle \mapsto \langle G', k \rangle$ avec si $G = (S, A)$ et $G' = (S, S^2 \setminus A)$.
La construction est bien en temps polynomial et G admet une clique de taille k ssi G' admet un ensemble stable de taille k .

- Si $(G, k) \in \text{Clique}$, soit $S' \subset S$ une clique de taille au moins k de G alors S' est un ensemble stable de G' de taille au moins k .
- Réciproquement, si $(G', k) \in \text{Stable}$ alors $S' \subset S$ un stable qui convient dans G' , c'est une clique de G .
- Clique étant NP-complet, on a bien Stable qui est NP-complet.

Théorème

Soit le problème **VertexCover** :

Entrée : $\langle G, k \rangle$

Sortie : est-ce qu'il existe un ensemble de sommets de taille k tq toute arête du graphe est adjacente à un de ses sommets.

VC est NP-complet

DÉMONSTRATION.

- $\text{VertexCover} \in \text{NP}$:
 - On propose d'utiliser les ensembles de S' comme certificat...
- $\text{Stable} \leq_p \text{VertexCover}$:
 - On prend $f : (G, k) \rightarrow (G, n - k)$.
 - G admet un stable de taille au moins k ssi il admet une couverture par sommets de taille au plus $n - k$ car $S' \subset S$ est un stable de G ssi $S \setminus S'$ est une couverture par sommets de G .

Enfin, comme Stable est NP-complet, on peut conclure.

Insérer un exemple. DÉMONSTRATION.

-
- **Indépendant Set $\leq_p$ VertexCover**
 $\langle G, k \rangle \rightarrow \langle G, n - k \rangle$
 On peut montrer que S' est un ensemble stable de $G = (S, A) \Leftrightarrow S \setminus S'$ est une couverture par sommet de G . Ainsi, $\langle G, k \rangle \in \text{IS} \Leftrightarrow \langle G, n - k \rangle \in \text{VC}$

6 Problèmes de recherche et problèmes d'optimisation

Définition

Un problème de recherche sur E est défini par une relation $\mathcal{R} \subset E \times S$ où S est un ensemble appelé ensemble de solution. Le problème serait alors décrit de la manière suivante;

Entrée : $x \in E$

Sortie : $y \in S$ tq. $\mathcal{R}(x, y)$

Proposition

On peut associer 2 types de problèmes de décision à un problème de recherche défini par $\mathcal{R} \subset E \times S$

- $f_{\exists} : x \in E \rightarrow \text{vrai}$ ssi $\exists y \in S$ tq. $\mathcal{R}(x, y)$
- $v : x \in E, y \in S \rightarrow \text{vrai}$ ssi $\mathcal{R}(x, y)$

Définition

Un problème d'optimisation est la donnée de $\mathcal{R} \subset E \times S$ et $c : S \rightarrow \mathbb{R}$ correspondant au problème suivant :

Entrée : $x \in E$

Sortie : $y \in S$ tq. $\mathcal{R}(x, y)$ qui minimise/maximise $c(y)$ parmi les y tq $\mathcal{R}(x, y)$.

Définition

On associe à un problème d'optimisation un problème de décision à seuil de la manière suivante.

OPT (mini ou max)

$\mathcal{R} \subset E \times S, c : S \rightarrow \mathbb{R}, x \in E \rightarrow y$ tq. $c(y) = \min_{z \in S} \{c(z) | \mathcal{R}(x, z)\}$ et $\mathcal{R}(x, y)$

Pb seuil (min) :

Entrée : $x \in E, s_0 \in \mathbb{R}$

Sortie : **true** ssi $\exists y \in \mathbb{R}, x \mathcal{R} y$ et $c(y) \leq s_0$

Exemple :

- **Problème d'optimisation :**

Entrée : $G = (S, A), s \in S, t \in S$

Sortie : renvoyer la description d'un plus court chemin de s à t dans G .

- **Problème de décision à seuil :**

Entrée : $G = (S, A, p), s \in S, t \in S, s_0 \in \mathbb{R}$

Sortie : **true** ssi il existe un chemin de s à t dans G de poids $\leq s_0$.